

Practice Exercise 1- Secure Data Protection Using Encryption in a Java REST API

Objective

Design and implement a secure Java REST API that protects sensitive information using encryption techniques. The application must ensure that confidential data remains protected while being stored and transmitted between client and server.

Background

Organizations often store sensitive personal and financial information that must be protected from unauthorized access. Failure to secure such information can result in identity theft, financial fraud, and privacy violations.

In this assignment, students will implement encryption to protect the following sensitive information:

Sensitive Fields

1. Passport Number

- Used for international travel identification.
- Must be protected from unauthorized access and misuse.

2. Bank Account Number

- Used for financial transactions and banking operations.

- Exposure can lead to financial fraud and unauthorized transactions.

3. Driving License Number

- Used as an official identity document.
- Leakage may result in identity theft and misuse of personal information.

To protect this information, encryption must be applied before storing or transmitting the data.

Assignment Requirements

Task 1: Create a Spring Boot Project

Create a Spring Boot REST application using Maven.

Required Dependency:

- Spring Web

Task 2: Design a Data Model

Create a model named **Citizen**, **Customer**, or **User** containing the following fields:

Field Name	Sensitive
Full Name	No
Passport Number	Yes
Bank Account Number	Yes

Driving License Number	Yes
------------------------	-----

Task 3: Implement Encryption Functionality

Develop a utility component that performs:

- Encryption of sensitive information
- Decryption of sensitive information

Requirements:

- Use a standard encryption algorithm.
 - Encrypt Passport Number, Bank Account Number, and Driving License Number.
 - Ensure encrypted values are not readable in plain text.
-

Task 4: Develop REST APIs

Create the following APIs:

API 1 – Save User Information

Purpose:

- Accept user details.
- Encrypt all sensitive fields.
- Return encrypted information.

API 2 – Retrieve User Information

Purpose:

- Accept encrypted user information.
 - Decrypt all sensitive fields.
 - Return original information.
-

Task 5: Validate Encryption

Verify that:

- Passport Number is encrypted before storage.
 - Bank Account Number is encrypted before storage.
 - Driving License Number is encrypted before storage.
 - Original values cannot be viewed directly after encryption.
-

Task 6: Test Using Postman

Perform the following tests:

Test Case 1

Submit original user information and verify that all sensitive fields are encrypted.

Test Case 2

Submit encrypted information and verify that all sensitive fields are correctly decrypted.

Capture screenshots of:

- Request payload

- Encrypted response
- Decrypted response

SOLUTION

```
secure-data-protection
|
|-- src
|   |-- main
|       |-- java
|           |-- com
|               |-- example
|                   |-- securedata
|                       |-- SecureDataApplication.java
|                       |-- controller
|                           |-- UserController.java
|                       |-- model
|                           |-- User.java
|                       |-- util
|                           |-- EncryptionUtil.java
|
|-- pom.xml
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.0</version>
  </parent>

  <groupId>com.example</groupId>
```

```
<artifactId>securedata</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
  <java.version>17</java.version>
</properties>

<dependencies>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

</dependencies>

<build>
  <plugins>

    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>

  </plugins>
</build>

</project>
```

SecureDataApplication.java

```
package com.example.securedata;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SecureDataApplication {

    public static void main(String[] args) {
        SpringApplication.run(SecureDataApplication.class, args);
    }
}
```

User.java

```
package com.example.securedata.model;

public class User {

    private String fullName;

    private String passportNumber;

    private String bankAccountNumber;

    private String drivingLicenseNumber;

    public User() {
    }

    public String getFullName() {
```



```
        return fullName;
    }

    public void setFullName(String fullName) {
        this.fullName = fullName;
    }

    public String getPassportNumber() {
        return passportNumber;
    }

    public void setPassportNumber(String passportNumber) {
        this.passportNumber = passportNumber;
    }

    public String getBankAccountNumber() {
        return bankAccountNumber;
    }

    public void setBankAccountNumber(String bankAccountNumber) {
        this.bankAccountNumber = bankAccountNumber;
    }

    public String getDrivingLicenseNumber() {
        return drivingLicenseNumber;
    }

    public void setDrivingLicenseNumber(String drivingLicenseNumber) {
        this.drivingLicenseNumber = drivingLicenseNumber;
    }
}
```

EncryptionUtil.java

```
package com.example.securedata.util;

import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class EncryptionUtil {

    private static final String SECRET_KEY =
        "1234567890123456";

    public static String encrypt(String data)
        throws Exception {

        SecretKeySpec key =
            new SecretKeySpec(
                SECRET_KEY.getBytes(),
                "AES");

        Cipher cipher =
            Cipher.getInstance("AES");

        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] encrypted =
            cipher.doFinal(data.getBytes());

        return Base64.getEncoder()
            .encodeToString(encrypted);
    }

    public static String decrypt(String encryptedData)
        throws Exception {

        SecretKeySpec key =
            new SecretKeySpec(
                SECRET_KEY.getBytes(),
```

```
        "AES");

    Cipher cipher =
        Cipher.getInstance("AES");

    cipher.init(Cipher.DECRYPT_MODE, key);

    byte[] decoded =
        Base64.getDecoder()
            .decode(encryptedData);

    return new String(
        cipher.doFinal(decoded));
    }
}
```

UserController.java

```
package com.example.securedata.controller;

import com.example.securedata.model.User;
import com.example.securedata.util.EncryptionUtil;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    @PostMapping("/saveUser")
    public User saveUser(
        @RequestBody User user)
        throws Exception {
```

```
user.setPassportNumber(  
    EncryptionUtil.encrypt(  
        user.getPassportNumber());  
  
user.setBankAccountNumber(  
    EncryptionUtil.encrypt(  
        user.getBankAccountNumber());  
  
user.setDrivingLicenseNumber(  
    EncryptionUtil.encrypt(  
        user.getDrivingLicenseNumber());  
  
return user;  
}
```

```
@PostMapping("/getUser")  
public User getUser(  
    @RequestBody User user)  
    throws Exception {  
  
    user.setPassportNumber(  
        EncryptionUtil.decrypt(  
            user.getPassportNumber());  
  
    user.setBankAccountNumber(  
        EncryptionUtil.decrypt(  
            user.getBankAccountNumber());  
  
    user.setDrivingLicenseNumber(  
        EncryptionUtil.decrypt(  

```

```
        user.getDrivingLicenseNumber()));  
  
    return user;  
}  
}
```

Run Application

mvn spring-boot:run

Application URL:

http://localhost:8080

Postman Test 1 – Encrypt Data

Request

POST

http://localhost:8080/saveUser

Body

```
{  
  "fullName": "Rahul Sharma",  
  "passportNumber": "P1234567",  
  "bankAccountNumber": "123456789012345",  
  "drivingLicenseNumber": "DL987654321"  
}
```

Sample Response

```
{  
  "fullName": "Rahul Sharma",  
  "passportNumber": "n7uN95Hn8Q4M2QnQ4kX3tw==",  
  "bankAccountNumber": "cA7Jj3jR3gL9e2Lw4VnY2A==",  
  "drivingLicenseNumber": "iD3P2qV9mN8sW4hL1cF7Qg=="  
}
```

Postman Test 2 – Decrypt Data

Request

POST

```
http://localhost:8080/getUser
```

Body

Use the encrypted values returned from Test 1.

Sample Response

```
{  
  "fullName": "Rahul Sharma",  
  "passportNumber": "P1234567",  
  "bankAccountNumber": "123456789012345",  
  "drivingLicenseNumber": "DL987654321"  
}
```